

CRISTOFF STAN

Senior Rust Software Engineer – Backend, Distributed Systems, Cloud

Location: Tallinn, Harjumaa, Estonia | Email: cristoffstan@gmail.com

LinkedIn: <https://www.linkedin.com/in/cristoff-s-692816413> | Phone Number: +372 58619311

PROFESSIONAL SUMMARY

Senior Software Engineer with 10+ years building B2B SaaS, e-commerce, and enterprise platforms, including 7+ years with **Rust 1.x, Tokio, Axum, Actix Web, PostgreSQL, Kafka, and AWS**; specializes in high-performance APIs, concurrent services, systems modernization, memory-safe architecture, and cloud delivery, turning latency-sensitive and reliability-critical workloads into secure, observable products with lower operating risk and stronger engineering velocity.

TECHNICAL SKILLS

- **Rust Engineering:** Rust 1.x, Cargo, Tokio, Axum, Actix Web, Warp, Rocket, tonic, Serde, Rayon, async/await, traits, generics, lifetimes, ownership, borrowing, macros, workspaces, crates, FFI and unsafe-code review.
- **Backend Development:** High-performance REST APIs, gRPC, GraphQL, WebSockets, asynchronous services, background workers, schedulers, streaming systems, distributed transactions, rate limiting, API versioning and service-to-service communication.
- **Architecture:** Microservices, modular monoliths, Domain-Driven Design, hexagonal architecture, clean architecture, event-driven systems, CQRS, transactional outbox, idempotent consumers, saga workflows and multi-tenant platforms.
- **Data and Messaging:** PostgreSQL, MySQL, MongoDB, DynamoDB, Redis, Elasticsearch/OpenSearch, Apache Kafka, RabbitMQ, AWS SQS, SQLx, Diesel, SeaORM, database migrations, indexing, query optimization and connection pooling.
- **Cloud Platforms:** AWS EKS, ECS, Lambda, EC2, API Gateway, RDS, DynamoDB, S3, SQS, SNS, EventBridge, IAM and CloudWatch; Azure AKS, Functions, Service Bus, Key Vault, Storage and Application Insights.
- **DevOps and Infrastructure:** Docker, Kubernetes, Terraform, Helm, GitHub Actions, GitLab CI, Jenkins, Argo CD, Linux, Nginx, container registries, infrastructure as code, canary releases and automated rollback.
- **Testing and Quality:** Rust test framework, cargo-nextest, rstest, proptest, criterion, mockall, Testcontainers, integration testing, property-based testing, fuzz testing, cargo-clippy, rustfmt, cargo-audit and dependency review.
- **Observability and Reliability:** OpenTelemetry, Prometheus, Grafana, Datadog, New Relic, Sentry, Elastic Stack, structured tracing, distributed traces, flame graphs, heap profiling, SLI/SLO design and incident response.
- **Security:** OAuth 2.0, OpenID Connect, JWT, RBAC, mTLS, secrets management, least-privilege IAM, input validation, supply-chain scanning, secure coding, threat modeling and audit logging.
- **Additional Languages:** Python 3.x, C++, TypeScript, JavaScript, Bash, SQL and limited Go for infrastructure utilities and interoperability with legacy services.
- **Senior Engineering Practices:** Technical leadership, architecture reviews, mentoring, code reviews, capacity planning, performance analysis, production readiness, root-cause analysis and stakeholder communication.

WORK EXPERIENCE

BEVY

Senior Software Engineer

(September 2021 – June 2026)

Remote, Romania

- Architected multi-tenant backend platforms using **Rust, Tokio, Axum, tonic, PostgreSQL, Redis, Kafka, and AWS**, establishing reusable service templates and production standards across several product teams.
- Designed asynchronous services for account management, billing, reporting, notification delivery, and document processing using bounded concurrency, structured cancellation, and explicit failure-handling strategies.
- Reduced p95 API latency by **43%** by replacing CPU-intensive Python services with Rust, optimizing SQL queries, eliminating unnecessary serialization, and introducing Redis-backed reference-data caching.
- Resolved production memory growth caused by retained `Arc` references, unbounded channels, and abandoned asynchronous tasks by adding lifecycle ownership, bounded queues, cancellation tokens, and workload-level metrics.
- Implemented event-driven workflows with **Kafka, transactional outbox records, idempotent consumers, dead-letter topics, and replay controls**, preventing duplicate updates during broker retries and partial failures.
- Migrated latency-sensitive Node.js and Python workloads into memory-safe Rust services while preserving existing API contracts through shadow traffic, compatibility tests, and incremental production cutovers.
- Built secure REST and gRPC APIs with **OAuth 2.0, OIDC, JWT, tenant-aware RBAC, request signing, rate limiting, audit trails, and secrets management** for customer and administrative operations.
- Improved production visibility with **tracing, OpenTelemetry, Prometheus, Grafana, flame graphs, and structured error contexts**, allowing engineers to correlate slow requests with database, queue, and downstream dependencies.
- Strengthened CI/CD using **GitHub Actions, Docker, Kubernetes, Terraform, cargo-nextest, cargo-audit, and canary deployments**, automating quality gates, image checks, schema validation, and rollback decisions.
- Mentored engineers on Rust ownership, async design, error modeling, API boundaries, performance profiling, and production incident analysis, improving code-review quality and service-level accountability.

Syscom Digital

Software Engineer

(October 2019 – August 2021)

Remote, Romania

- Developed commerce and operations services using **Rust, Actix Web, Tokio, PostgreSQL, Redis, Kafka, and AWS**, supporting checkout, payment reconciliation, inventory, and customer-service workflows.
- Built asynchronous payment and webhook processors with signature verification, idempotency records, controlled retries, and provider-specific adapters that isolated unreliable external integrations.
- Reduced recurring payment-processing failures by **31%** by introducing correlation IDs, dead-letter handling, replayable events, transactional status updates, and clearer classification of provider errors.
- Resolved throughput degradation caused by lock contention and oversized critical sections by replacing shared mutable state with message passing, partitioned caches, and shorter transactional operations.

- Implemented REST and gRPC integrations for payment gateways, shipping providers, and internal services using **Serde, Reqwest, tonic, and strongly typed domain models**.
- Improved PostgreSQL performance through composite indexes, prepared statements, query-plan analysis, batched writes, and connection-pool tuning for traffic-heavy order and inventory endpoints.
- Replaced selected Python workers with Rust consumers for high-volume event processing, reducing container resource usage while maintaining compatibility with existing Kafka schemas.
- Expanded delivery automation with **Jenkins, GitLab CI, Docker, cargo test, Clippy, security audits, and staged deployments** across development, QA, and production environments.
- Collaborated with product owners, QA engineers, and operations teams to translate ambiguous commerce requirements into explicit domain rules, failure scenarios, and measurable acceptance criteria.

Soft Build

Software Developer

(June 2016 – July 2019)

Remote, Romania

- Built backend services and command-line utilities using **Rust, Python, C++, PostgreSQL, MySQL, Redis, and RabbitMQ** for e-commerce, booking, reporting, and administration products.
- Improved batch-processing throughput by **41%** by moving parsing and pricing calculations from Python into parallel Rust workers while preserving existing database schemas and output contracts.
- Developed REST APIs and asynchronous workers for inventory, appointments, pricing, reporting, and customer-management modules using clear service, repository, and domain boundaries.
- Resolved crashes and corrupted intermediate files in legacy C++ utilities by rebuilding critical parsers in Rust with validated input models, explicit error types, and atomic output replacement.
- Implemented background processing for email, report generation, import validation, and external synchronization with bounded queues, retry limits, deduplication keys, and failure reporting.
- Added unit, integration, property-based, and benchmark tests around parsers, pricing calculations, persistence adapters, and critical API behavior to reduce regression and performance risk.
- Containerized Rust and Python services with **Docker**, standardized Linux configuration, and added Jenkins pipelines for repeatable builds, database migrations, and deployment verification.
- Worked with senior engineers on SQL execution plans, memory profiling, concurrency reviews, release automation, and production incidents across data-intensive business applications.

Decima Digital

Software Developer

(November 2015 – April 2016)

Remote, Estonia

- Delivered backend functionality using **Python, C++, Django, PostgreSQL, MySQL, JavaScript, and Linux** for customer-facing websites, administration tools, file-processing services, and reporting workflows.
- Built service utilities for content retrieval, form processing, file uploads, notification routing, data imports, and scheduled report generation with explicit validation and recoverable error handling.
- Fixed production defects involving malformed CSV files, encoding failures, null values, partial uploads, and inconsistent database records by strengthening input checks and transactional updates.
- Improved slow reporting operations by reviewing ORM-generated queries, adding targeted indexes, using bulk writes, and replacing repeated record loops with batched transformations.
- Developed small C++ command-line utilities for CPU-intensive text and file processing while documenting memory ownership, error codes, and operational recovery procedures.
- Participated in Linux deployment preparation, smoke testing, log review, backup verification, and rollback planning, helping reduce release risk across multiple client applications.
- Collaborated with designers, project managers, and senior engineers to convert business requirements into maintainable backend modules, responsive interfaces, and realistic implementation estimates.

Borna Technology
Software Developer Intern
(September 2014 – August 2015)
Tallinn, Estonia

- Assisted senior engineers with **Python, C++, Django, JavaScript, PostgreSQL, MySQL, Git, and Linux** tasks across internal tools, client websites, and data-driven applications.
- Built reusable backend utilities, template components, validation scripts, and data-import helpers while learning structured debugging, code review, and disciplined version-control practices.
- Supported development of form handlers, administration pages, file-processing tools, database queries, and basic REST-style endpoints under guidance from experienced engineers.
- Investigated invalid submissions, memory errors, Python exceptions, SQL failures, and inconsistent JSON responses by reproducing problems and documenting reliable diagnostic steps.
- Contributed to QA checks, deployment notes, basic test automation, and technical documentation, establishing a foundation in maintainable systems engineering and production support.

EDUCATION

Tallinn University

Bachelor's Degree in Computer Science,
September 2011 – May 2014

CERTIFICATIONS

- **AWS Certified Solutions Architect – Associate**
- **AWS Certified Developer – Associate**
- **Certified Kubernetes Application Developer — CKAD**
- **HashiCorp Certified: Terraform Associate**
- **Linux Foundation Certified System Administrator — LFCS**

PROJECTS

High-Throughput Event Processing Platform

- Designed a distributed processing platform using **Rust, Tokio, Axum, Kafka, PostgreSQL, Redis, Kubernetes, and AWS EKS** for payment events, order updates, notifications, and operational reconciliation.
- Implemented partition-aware consumers, idempotency keys, transactional outbox publishing, dead-letter topics, event replay, schema validation, backpressure, and graceful shutdown for predictable processing during traffic spikes.
- Added distributed tracing, consumer-lag dashboards, benchmark suites, load tests, and automated canary deployment checks to validate throughput, latency, and correctness before production promotion.

Legacy Service Modernization Program

- Led the migration of performance-sensitive **Python, Node.js, and C++ services** into memory-safe Rust workloads while retaining existing REST contracts, event schemas, and database integrations.
- Used shadow traffic, differential testing, property-based tests, flame graphs, and staged traffic shifting to detect behavioral mismatches and performance regressions before each production cutover.

- Introduced reusable Rust workspace templates, centralized error handling, tracing conventions, secure dependency policies, and CI quality gates to standardize development across new services.